UCS503P - Software Engineering Project

MATES

A Taste Based Social Discovery Platform

Submitted by

Tulika Jain 1024160110
Mukul 1024160124
Pranav Tiwari 1024160127

BE Third Year, CSE

Under the Guidance of

Anushka Ma'am

Mid-Semester Evaluation

Computer Science and Engineering Department
Thapar Institute of Engineering and Technology, Patiala

Contents

1 Introduction **2**
- 1.1 Background and Context . . . 2
  - 1.1.1 Problem Statement . . . 2
- 1.2 Project Objectives . . . 2

2 System Architecture **3**
- 2.1 High Level Design . . . 3
- 2.2 Implementation Details . . . 8
  - 2.2.1 The Compatibility Engine . . . 8
  - 2.2.2 Feature Modules . . . 8

3 UI/UX Design **9**
- 3.1 Design Concept . . . 9
- 3.2 Typography . . . 9
- 3.3 Core Components . . . 9
- 3.4 Feature to Screen Mapping . . . 10

4 Technology Stack **11**

5 Results and Evaluation **12**
- 5.1 Testing Methodology . . . 12
- 5.2 Evaluation Metrics . . . 12
- 5.3 Analysis . . . 13

6 Conclusion **14**
- 6.1 Summary of Work . . . 14
- 6.2 Key Findings . . . 14
- 6.3 Future Work and Recommendations . . . 14
- 6.4 Final Remarks . . . 15

Bibliography **16**

Chapter 1

Introduction

1.1 Background and Context

Existing social platforms are typically organized around a single content category such as music, photos, or short video, or around a follower based model that optimizes for reach rather than genuine compatibility. A person's taste is usually spread across several disconnected apps, one for music, one for movies, one for books, with no unified representation of who they actually are. MATES addresses this by translating cross domain taste similarity into a measurable, explainable compatibility score, delivered through a working mobile application.

1.1.1 Problem Statement

Users who want to find people who genuinely share their taste face four recurring pain points:

- **Fragmentation:** taste signals for music, film, and books live in separate, disconnected apps.
- **Opaque recommendations:** existing "suggested for you" features rarely explain *why* two users were matched, which erodes trust.
- **Follower centric design:** one directional following does not guarantee any shared interest, producing noisy, low relevance feeds.
- **Weak niche discovery:** users with specific or unusual combinations of interests struggle to find others who share that exact combination.

1.2 Project Objectives

1. **Objective 1:** Deliver a working, end to end core loop: sign up, import taste, view ranked matches, see why the match was made, connect, and chat, all against live data.
2. **Objective 2:** Make every compatibility score explainable by surfacing the specific shared items and divergence highlights behind it, rather than an opaque ranking.
3. **Objective 3:** Extend the core loop with social features (content sharing, communities, and real time chat) so the app supports ongoing engagement rather than a single match event.

Chapter 2

System Architecture

2.1 High Level Design

The system follows a mobile client talking to a REST backend, which in turn talks to external taste data providers and to the Claude API for narrative generation. The core workflow is: a user authenticates, imports taste keywords across music, movies, and books, and the backend resolves each item against a live external provider. The Compatibility Engine then scores the user against a candidate pool using a weighted blend of similarity measures, and returns a ranked, explainable Discovery feed. A connection request that gets accepted unlocks chat, and an AI generated conversation starter is offered alongside it. Users can also post Content Drops from their own taste, react to other people's drops, and join tag based Communities suggested from their taste vector. Figures 2.1 to 2.4 document this behaviour at increasing levels of detail.

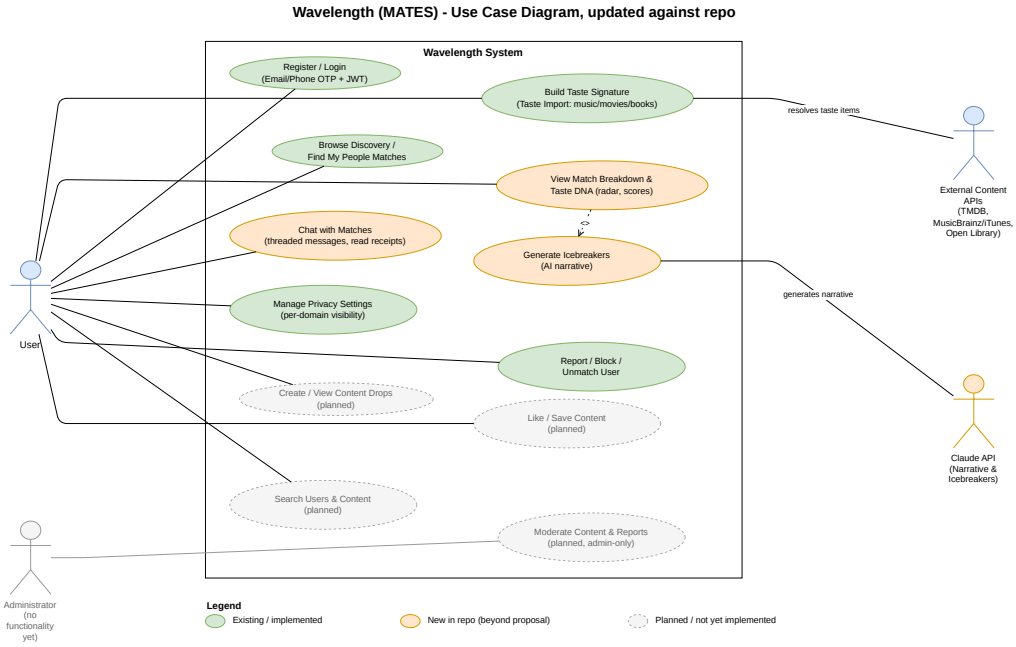


Figure 2.1: Use case diagram of the MATES system.

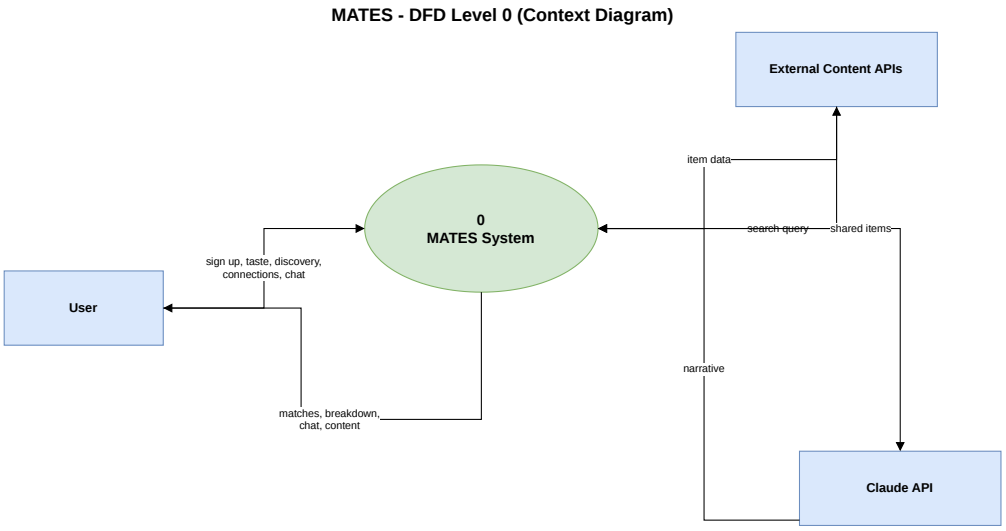


Figure 2.2: Data flow diagram, Level 0, context diagram.

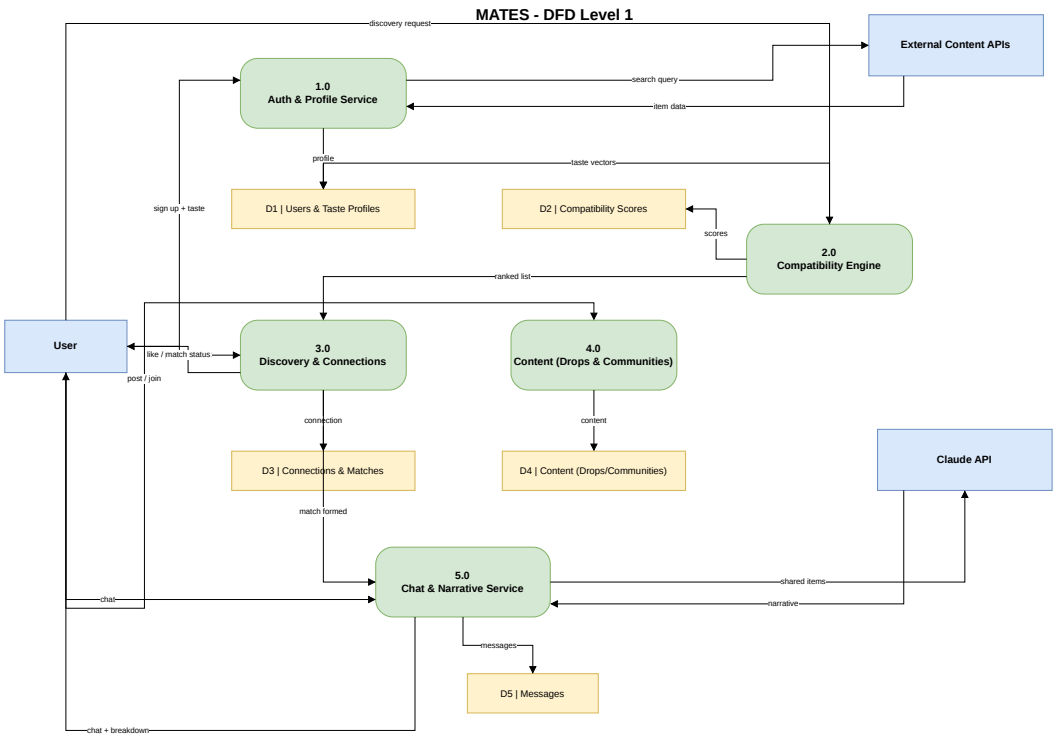


Figure 2.3: Data flow diagram, Level 1, showing the backend processes and data stores that make up the system.

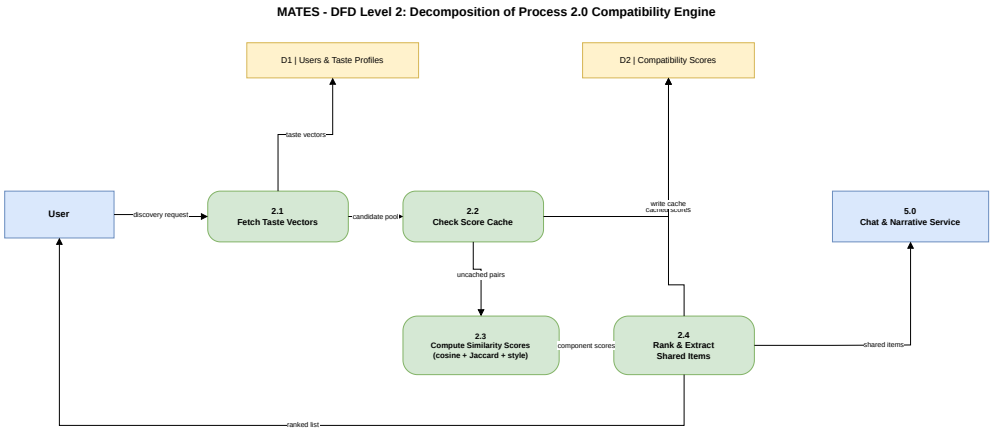


Figure 2.4: Data flow diagram, Level 2, decomposition of the Compatibility Engine process.

2.2 Implementation Details

2.2.1 The Compatibility Engine

The compatibility engine lives in its own dependency free module, with no database, HTTP, or framework code, so the scoring algorithm can be developed and unit tested in isolation. It computes, for a pair of users:

$$\text{score} = 0.45 \times \text{cosine} + 0.35 \times \text{weighted Jaccard} + 0.20 \times \text{style alignment}$$

where cosine similarity captures overall taste shape, weighted Jaccard captures literal shared favourites, with each shared item weighted by inverse popularity so two people who both love an obscure film score far higher than two who both love a blockbuster, and style alignment captures era and mood overlap. Domain weights across music, movies, and books are not fixed at a third each: a domain counts only when both users have data in it, scaled by data depth and each user's stated priority, so an unconnected domain cannot silently drag a score toward zero. Every score returned by the engine carries the concrete shared items that produced it, plus divergence highlights, the distinctive things one person loves that the other has never touched, which double as conversation prompts for the icebreaker feature.

2.2.2 Feature Modules

- **Auth:** email and phone OTP, JWT issuance, and replay and brute force protection.
- **Onboarding:** name, age, city, bio, and stated intent (dating, friends, or both).
- **Taste import:** live search and add across all three launch domains.
- **Discovery:** a draggable, compatibility ranked card stack that is aware of the user's stated intent, with filters for minimum match and age.
- **Connections:** an opt in, bidirectional request and accept flow. A like sends a connection request; chat unlocks only once the other person accepts, and a mutual like accepts instantly.
- **Breakdown:** a score ring, per domain scores, the Taste DNA radar (five axes derived from the data, never self reported), an AI written narrative, and a share/differ toggle.
- **Narrative and icebreakers:** generated via the Claude API when an API key is configured, with a written in code fallback that still names real, concrete shared titles [Ant26].
- **Content Drops:** short posts built from a user's own taste items, with like and save reactions, an optional community tag, and a "you have this too" flag when a drop matches the viewer's own profile.
- **Communities:** tag based rooms with join and leave, suggestions ranked by the user's own taste tags, a member list, and a per community drop feed.
- **Chat:** threaded messages, read receipts, and taste based icebreakers, including seeded demo profiles that reply instantly for testing.
- **Add a profile:** a tool to build synthetic people by hand so they join the matching pool at once, used to seed a demoable Discovery feed.
- **Appearance:** dark, light, and system theme, persisted across sessions.
- **Safety:** block, report, and unmatch.
- **Privacy:** per domain visibility, set to full, aggregate, or hidden.

Chapter 3

UI/UX Design

3.1 Design Concept

MATES is about the texture of what someone loves, so the interface stays dark and low chroma and lets artwork and type carry the colour. Three accent hues are reserved for the three taste domains, music, movies, and books, and are never used decoratively for anything else, so a colour always means the same thing wherever it appears in the app. A light theme mirrors the same geometry and type scale, with the domain accents darkened slightly so they stay legible on a pale background; the user can switch between dark, light, and system at any time from Settings.

Name	**Hex**	**Usage**
Ink	`#0E0D12`	App background, warm near black
Surface	`#17161D`	Cards and input fields
Surface High	`#232129`	Elevated surfaces, disabled states
Stroke	`#2E2B36`	Borders and dividers
Cream	`#F6F3EC`	Primary text
Muted	`#8E8A9C`	Secondary text
Faint	`#5E5A6B`	Tertiary text, low emphasis icons
Music accent	`#6FE3C4`	Music domain, primary actions, “like”
Movie accent	`#F2A65A`	Movie domain
Book accent	`#B08CF0`	Book domain
Danger	`#E5687A`	Destructive actions, “pass” or report

Table 3.1: Dark theme colour palette

3.2 Typography

The base Material 3 type scale is overridden with a tighter, more compact scale, tuned for a card heavy, data dense interface. Display and headline styles use negative letter spacing so large numerals and titles feel tight rather than airy, and body styles use a slightly taller line height for comfortable reading in chat and captions.

3.3 Core Components

- **Score Badge:** renders a compatibility score with a colour that shifts along the palette as the score rises, from a faint grey at low compatibility to the music accent teal at a strong match.
- **Taste DNA Radar:** a five axis radar chart, built with `fl_chart`, visualising a user’s taste profile.

Style	Size	Weight
Display Small	36	700
Headline Medium	27	700
Title Large	20	600
Title Medium	17	600
Title Small	15.5	600
Body Medium	15	400
Body Small	13.5	400
Label Small	11.5	600
Match score numerals	46	700

Table 3.2: Type scale. Match score numerals additionally use tabular figures so the digits do not jitter as a score animates.

- **Tab Shell:** the five tab bottom navigation shared across Discover, Drops, Rooms, Matches, and Taste.
- **Entrance:** a staggered fade and slide in animation applied to list items as they load, used across Discovery, Drops, and Communities.
- **Page Shell:** a consistent page padding and scroll wrapper used by every tab.
- **Mates Mark:** the animated app mark shown on the splash screen while the session is restored.

3.4 Feature to Screen Mapping

Screen	What It Does	Concept Exercised
Sign In / Verify	Email/phone OTP request and verification	Authentication: OTP issuance, JWT, brute force protection
Onboarding	Collects name, age, city, bio, and intent	Input validation and user profile modeling
Taste / Taste Search	Live search and add across music, movies, books	External API integration (TMDB, MusicBrainz/iTunes, Open Library)
Discover	Compatibility ranked, swipeable card stack with filters	Consuming a recommendation and matching algorithm
Compatibility Breakdown	Score ring, per domain scores, Taste DNA radar, narrative	Data visualisation and AI generated content
Drops	Compose and share taste items, like/save, tag a community	Content feed and social interaction state
Rooms (Communities)	Join/leave, suggestions, member list, per room feed	Ranking by tag overlap
Matches	Pending requests and active conversations	Connection state machine: request, accept, chat unlock
Chat	Threaded messages, read receipts, AI icebreakers	Messaging model and AI integration
Settings	Appearance and per domain privacy	User preference persistence
Requests	Inbox of pending connection requests	Connection request and accept workflow
Add a Profile	Builds a synthetic profile for the matching pool	Data seeding for cold start mitigation

Table 3.3: Mapping of each screen to what it does and the concept it exercises

Chapter 4

Technology Stack

Layer	Choice	Notes
Mobile app	Flutter 3.47 / Dart 3.13 [Goo26]	Riverpod for state, go_router for routing, dio for HTTP
Data visualisation	`fl_chart`	Powers the Taste DNA radar chart
Backend API	Node.js 20+, TypeScript, Fastify 5 [Fas26]	REST endpoints, TypeBox schemas, generated OpenAPI contract
Database	PostgreSQL [Pos26]	PGlite (embedded WASM Postgres) for local development [Ele26], real Postgres in production
Auth	Email/phone OTP + JWT	Self implemented, no third party auth dependency
AI narrative	Claude API [Ant26]	Generates match narratives and icebreakers, with a code based fallback
Taste providers	MusicBrainz/iTunes, TMDB/Wikidata [The26], Open Library [Int26]	Every domain resolves with zero required API keys
Testing	vitest (server), flutter_test (app)	Server suite covers auth, compatibility, connections, drops, and communities
Contract	Generated OpenAPI document	Shared source of truth between the app and the server

Table 4.1: Technology stack

Domain	Primary provider	Fallback
Music	MusicBrainz	iTunes
Movies	TMDB (if a key is configured)	Wikidata SPARQL
Books	Open Library	no fallback needed

Table 4.2: Taste data providers, all usable with zero required configuration

Chapter 5

Results and Evaluation

5.1 Testing Methodology

- **Unit Testing:** the server test suite, built with `vitest`, covers authentication, the compatibility engine, connections, drops, and communities.
- **Integration Testing:** `api_integration_test.dart` exercises the real Dart client against a running server instance. It skips itself when the server is down, so `flutter test` stays green either way.
- **Static Analysis:** `flutter analyze` is run on the client with zero outstanding issues.
- **User Testing:** not yet run. A pilot with real users, supplemented by synthetic profiles to mitigate cold start, is planned for the next stage.

5.2 Evaluation Metrics

The following metrics are used to evaluate matching quality once a pilot runs. Several are already instrumented in the schema, since the underlying events are timestamped as they happen.

Metric	**Status**
Match to connection rate	Instrumented: swipe and connection request events are timestamped
Connection acceptance rate	Instrumented: connection request status and response time are stored
Taste Signature completion rate	Instrumented: profile completeness is tracked per user
Match explanation density	Instrumented: shared highlights are stored per compatibility pair
Explainability satisfaction (1 to 5)	Needs an in app rating prompt, not yet built
Reliability / uptime	Needs a deployment target with monitoring, not yet built
Server unit tests	Passing, current suite covers auth, compatibility, connections, drops, communities
`flutter analyze` issues	Zero issues, as reported by the client build

Table 5.1: Evaluation metrics and current status

5.3 Analysis

Did the team meet its objectives? Objective 1 is met: the core loop, sign up, import taste, see who matches, see why, connect, and chat, works end to end against live data. Objective 2 is met by construction, since the compatibility engine cannot return a score without also returning the shared items and divergence highlights behind it. Objective 3 is met: Content Drops, Communities, and Chat are all built and reachable from the main tab shell, extending the app beyond a single match event into ongoing engagement.

Design decisions and how they were resolved. The backend uses Fastify with TypeScript so the API and the generated contract share one language end to end. PostgreSQL was chosen because the data is fundamentally relational, pairs, requests, and memberships, and database constraints catch real bugs such as an ordered pair violation or a self swipe. Authentication is a self hosted OTP and JWT flow, which keeps the whole project runnable without any third party accounts or keys. Profile portraits are currently generated rather than uploaded, so the app has a complete, demoable visual identity for every user without needing an upload pipeline yet. On the toolchain side, iOS and Android builds are currently blocked by an incomplete Xcode installation and a missing Android SDK on the development machine, so a web target was added to the Flutter project purely to verify the UI in the meantime.

Chapter 6

Conclusion

6.1 Summary of Work

MATES implements a cross domain Taste Signature feeding an explainable compatibility score, delivered as a working mobile application with a tested backend engine. The core discovery to chat loop is complete and running against live external data sources, and the social layer around it, Content Drops, Communities, and Chat, is built and reachable from the main navigation.

6.2 Key Findings

- **Finding 1:** An explainable compatibility engine, cosine similarity, inverse popularity weighted Jaccard overlap, and style alignment combined at fixed weights, is straightforward to keep dependency free and unit testable, which made it safe to build social features such as Chat and Drops on top of it with confidence.
- **Finding 2:** Choosing self hosted or key free equivalents for authentication and taste data providers kept the whole stack runnable with zero external accounts, which made local development and demoing straightforward.
- **Finding 3:** Instrumenting evaluation events, swipes, connection requests, and profile completeness, from the start means the metrics needed for a pilot are already collectible; what remains is running the pilot itself and building the rating and monitoring surfaces around it.

6.3 Future Work and Recommendations

1. Continue building out the content part of the app, Taste Rooms, Blind Taste Mode, Collaborative Blends, and a Weekly Recap, for which schema support already exists.
2. Implement in app Search across users and content, and an Admin/Moderation dashboard for reviewing reports and flagged content.
3. Build the notification layer for connections, likes, and new matches.
4. Add an in app explainability rating prompt so the 1 to 5 clarity metric can be collected from real users.
5. Move the compatibility score from a per request cache to a nightly precompute job once user volume justifies it.
6. Add Spotify OAuth for listening history and audio features, to complete the music vector's sonic fingerprint.

7. Implement a real photo upload pipeline; the schema and moderation columns already exist, and portraits are currently generated rather than uploaded.
8. Replace console logged OTPs with a real SMS or email delivery provider.
9. Complete iOS and Android toolchain setup so the app can move off the temporary web target and onto real devices.
10. Stand up a CI/CD pipeline, automated build, backend tests, linting, and staging deployment, and a real deployment target so reliability and uptime can be measured.
11. Run the planned pilot with real users, supplemented by synthetic profiles, and use it to validate the evaluation metrics end to end.

6.4 Final Remarks

The project validates its core engineering bet, that a lightweight, explainable, cross domain compatibility score is enough to drive a working discovery to connection loop without needing a research grade matching model, and that the same foundation supports an active social layer on top of it. The remaining work is concentrated in deeper content features, search, moderation tooling, and deployment, rather than in the matching core itself, which is a reasonable place to be at the mid semester stage.

Bibliography

[Ant26] Anthropic. *Claude API Documentation*. 2026. URL: https://docs.claude.com (visited on 09/16/2026) (cit. on pp. 8, 11).

[Ele26] ElectricSQL. *PGlite: Embedded Postgres in WASM*. 2026. URL: https://pglite.dev (visited on 09/16/2026) (cit. on p. 11).

[Fas26] Fastify Team. *Fastify: Fast and Low Overhead Web Framework for Node.js*. 2026. URL: https://fastify.dev (visited on 09/16/2026) (cit. on p. 11).

[Goo26] Google. *Flutter Documentation*. 2026. URL: https://docs.flutter.dev (visited on 09/16/2026) (cit. on p. 11).

[Int26] Internet Archive. *Open Library Developer Center*. 2026. URL: https://openlibrary.org/developers (visited on 09/16/2026) (cit. on p. 11).

[Pos26] PostgreSQL Global Development Group. *PostgreSQL Documentation*. 2026. URL: https://www.postgresql.org/docs/ (visited on 09/16/2026) (cit. on p. 11).

[The26] The Movie Database (TMDB). *TMDB API Documentation*. 2026. URL: https://developer.themoviedb.org/docs (visited on 09/16/2026) (cit. on p. 11).